Name: Jared Rosenblum
Date: May 20, 2026
Course: IT FDN 110 GP – Foundations of Python Programming

Assignment 05 - Creating a Python Program

Introduction

This document describes the steps taken to create a Python program which provides the user with a menu of options to register a student for a course. The program allows the user to register students by inputting the student's first and last name, along with the course they're registering for. As long as this option is chosen, the user can continue to register as many students as needed. The user can then review the registration data and/or save it to an enrollment .json file. When finished, there is an option for the user to exit the program. Additionally, all of the registration data previously entered will still be available to review upon exiting and/or rerunning the program.

Creating and Executing the Python Script

Before creating the Python program, an Integrated Development Environment (IDE) is recommended to assist in writing, testing, managing and debugging the underlying code.[1] One of the more intuitive IDE's is PyCharm, which can be downloaded and installed at the following link - https://www.jetbrains.com/pycharm/download.

Once PyCharm is installed and a new project and .py file are created, the following steps can be taken to develop and execute a script that will display a menu with options to begin inputting individual student registration details, view the data that's been entered, save it to an enrollment .json file, and exit the program.

1. Begin by importing the JSON module which is a built-in module in Python. JSON is short for JavaScript Object Notation and is a lightweight data interchange format that is easy for people to read and write, and is also easy for machines to parse and generate. The files consist of key-value pairs organized in { } for objects and [ ] for arrays.[2] The module can be imported by writing the syntax *import json* at the beginning of the program.

2. Define and set constants to be used as part of the program, which will remain unchanged.[3] The constants will be a list of menu options displayed to the user when they go through and run the program. This includes four options to 1) Register a student for a course, 2) Show current data, 3) Save data to a file and 4) Exit the program. Additionally, FILE_NAME is set up to indicate the name of the .json file that will be written to and saved down as part of menu option 3.

As a standard convention to help identify a constant in the script, it should be written in uppercase letters.[4] The constants used as part of the script are as follows:

```
# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-----------------------------------------
'''
# Define the Data Constants
FILE_NAME: str = "Enrollments.json"
```

3. Define and set variables - “menu_choice”, ”student_first_name”, “student_last_name”, and “course_name” - that will store values input by the user. By using the input() function as part of the “menu_choice” variable, the user will be prompted to select one of the options from the menu. If electing to register a student for a course, each of the other variables would be set up to allow the user to input a student’s first name, last name, and the course that they’re registering for.

   To help organize the data input by the user, set up a dictionary and a list variable, called “student_data” and “students”, respectively. The “student_data” variable would hold one row of the student data input by the user, which would encompass the students first and last name and course that they’re registered for. The “students” variable represents a dictionary inside of a list, or a table, that would hold all of the rows created as part of “student_data.”

   Finally, set up the variable “file”, which will be used to write the content entered as part of the program to a separately created enrollments .json file.

```
# Define the Data Variables and constants
student_first_name: str = ''  # Holds the first name of a student entered by the user.
student_last_name: str = ''  # Holds the last name of a student entered by the user.
course_name: str = ''  # Holds the name of a course entered by the user.
student_data: dict = {}  # one row of student data (TODO: Change this to a Dictionary)
students: list = []  # a table of student data
file = None  # Holds a reference to an opened file.
menu_choice: str  # Hold the choice made by the user.
```

4. For clarification and ease of review in reading the Python script, indicate the data types that are being used.[4] In this case, the constants and variables are made up of strings, lists, and dictionaries. The “None” indicates that a variable is not assigned a value yet.

These are often used as a place holder for data that you plan to add later.[5] The data types are outlined as part of the syntax in steps 2 and 3 above.

5. In order to extract and read the Enrollment.json file to work with, use the open() function, with the “r” option. This accesses the contents of the file without modifying them.[6] The json.load(file) method parses the JSON data from FILE_NAME into a Python list of dictionaries.[7]

   Surrounding the code is a structured error handling method called try-except. Error handling improves the script by managing errors that you may not have control over any other way. You can actively plan for mistakes and make the process of recovering from errors smoother. The Try-except construct is an organized way to trap errors, while providing general and user friendly error messages.[8]

   In the case of opening the file, a try-except block contains both the FileNotFoundError and Exception class to help address any errors.. The e.__doc__ syntax is an attribute that will print the documentation string of the exception type, providing a brief description of the exception type and what it signifies. The type(e) syntax will print the type of exception object.[9]

   The finally block runs after the try and except blocks finish, ensuring the file is always closed properly.[10]

```
# When the program starts, read the file data into a list of lists (table)
# Extract the data from the file
try:
        file = open(FILE_NAME, "r")
        students = json.load(file)
except FileNotFoundError as e:
        print("File must exist before running this script!\n")
        print("-- Technical Error Message -- ")
        print(e, e.__doc__, type(e), sep='\n')
except Exception as e:
        print("There was a non-specific error!\n")
        print("-- Technical Error Message -- ")
        print(e, e.__doc__, type(e), sep='\n')
finally:
        if file is not None and file.closed == False:
            file.close()
```

6. Set up a “while loop” that will display the MENU data constant for the user, asking them to select one of the four options. The while loop will execute a set of instructions repeatedly as long as a certain condition remains true.[11] Based on the syntax, the program will continuously display the menu and related options for the user to select. It’s important to note that the loop will run indefinitely unless a break is included as part of the syntax to end the loop prematurely.[12] In this case, option 4 will allow the user to exit the program via the break statement. This will be explained in further detail when discussing the procedures for menu option 4 below.

Syntax:

```
while True:
    print(MENU)
    menu_choice = input("What would you like to do: ")
```

Results upon running program:

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do:
```

7. In order to control the flow of the program, create conditional “if” and “elif” statements associated with each of the menu options.[13] Additionally, write the necessary code as part of each statement in order to appropriately execute the option selected. Below is a breakdown of the syntax and results for each of the four menu options:

    - ***Menu Option 1*** - Register a student for a course

      If the user selects menu option 1, prompt the user to input a student’s first name, last name, and course they’re registered for. Add the data to a dictionary using the student_data variable, and then add that dictionary to the students list to create a table of data (a dictionary inside of a list). Each of the student registration variables (first name, last name, course name) are elements of a dictionary structured as a key-value relationship. The keys are the “FirstName”, “LastName”, “CourseName”, while the values are the string variables that will ultimately be input by the user. The elements all sit inside curly brackets { }, indicating that they’re part of a dictionary.[14]

      Similar to step 5 above in reading the JSON file, a try-except error handling method is used. Both the ValueError and Exception classes are used to catch and handle specific input errors on the student first name and last name, as well as any other exceptions. The e.__doc__ syntax attribute will provide a brief description of the exception type and what it signifies. The type(e) syntax will print the type of exception object.[9]

```
while True:
    print(MENU)
    menu_choice = input("What would you like to do: ")
    if menu_choice == "1":
        try:
            student_first_name = input("Enter the student's first name: ")
            if not student_first_name.isalpha():
                raise ValueError("First name can only have alphabetical characters!")
            student_last_name = input("Enter the student's last name: ")
            if not student_last_name.isalpha():
                raise ValueError("Last name can only have alphabetical characters!")
            course_name = input("Please enter the name of the course: ")
            student_data = {"FirstName": student_first_name,
                            "LastName":student_last_name,
                            "CourseName":course_name}
            students.append(student_data)
            print(f"You have registered {student_first_name} {student_last_name} for {course_name}.")
        except ValueError as e:
             print("-- Technical Error Message -- ")
             print(e, e.__doc__, type(e), sep='\n')
        except Exception as e:
             print("There was a non-specific error!\n")
             print("-- Technical Error Message -- ")
             print(e, e.__doc__, type(e), sep='\n')
        continue
```

- ***Menu Option 2*** - Show current data

  Use the print() function to display the student set up previously.

  Concatenate the data entered using an f-string (introduced as part of Python 3.6). An f-string allows you to embed expressions directly inside string literals by prefixing the string with "f." In this case, elements created as part of the student_data dictionary are accessed and used as expressions inside curly-braces {}. These are then evaluated and their values are inserted into the string.[15]

  The print("-"*50) is for presentation purposes only, displaying dashes around the data.

```
# Present the current data
    elif menu_choice == "2":
        print("-"*50)
        for student in students:
            print(f"Student {student["FirstName"]} {student["LastName"]} is enrolled in {student["CourseName"]}")
        print("-"*50)
        continue
```

  As an example, as part of option 2, if the user inputs the details to register Vic Vu for Python 100 and Jane Doe for Python 200 the following will be displayed:

```
What would you like to do: 2
-------------------------------------------------
Student Vic Vu is enrolled in Python 100
Student Jane Doe is enrolled in Python 200
-------------------------------------------------
```

- ***Menu Option 3*** - Save data to a file

  Leveraging the constants and variables set up in steps 2 and 3 above, write the content of the students table by first opening the FILE_NAME (Enrollments.json) file in write mode using the open() function. Include a “w” as part of the code to indicate that you want to write to the file.[16]

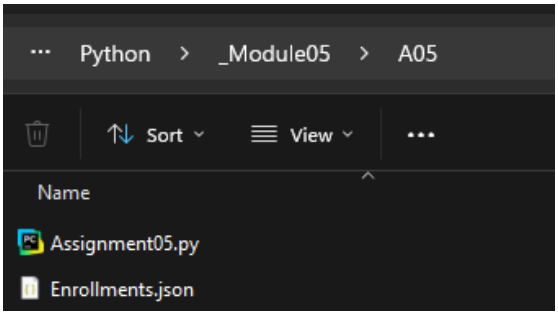
  Then using the json.dump() function, write the first name, last name, and course name data captured as part of the students table into the file.[17] Finally, close the file using the close() function.[16]

  To help handle errors, surround the code with the try-except construct. Use both the TypeError and Exception classes to handle any errors that could arise from going through and closing the file. The e.__doc__ syntax is an attribute that will print the documentation string of the exception type, providing a brief description of the exception type and what it signifies. The type(e) syntax will print the type of exception object.[8,9]

  The finally block runs after the try and except blocks finish, ensuring the file is always closed properly.[10]

```
# Save the data to a file
    elif menu_choice == "3":
        try:
            file = open(FILE_NAME, "w")
            json.dump(students, file)
        except TypeError as e:
            print("Please check that the data is a valid JSON format\n")
            print("-- Technical Error Message -- ")
            print(e, e.__doc__, type(e), sep='\n')
        except Exception as e:
            print("-- Technical Error Message -- ")
            print("Built-In Python error info: ")
            print(e, e.__doc__, type(e), sep='\n')
        finally:
            if file is not None and file.closed == False:
                file.close()
        continue
```

This will write to the file titled Enrollments.json in the same folder location that the .py file is under. The file will include all of the registration data that was entered by the user as part of menu option 1.



- ***Menu Option 4*** - Exit the program

  In order to break or stop the “while loop” created at the start of the program, provide the user with menu option 4. This will stop the program and let the user know that the “Program Ended.”

  Add one final ‘else’ clause in case the user enters a number other than 1-4. Let the user know that their choice was invalid and to enter one of the available options. Otherwise the program will continue to run and the user will be unaware if they entered an inapplicable option.

  Syntax:

```
# Stop the loop
    elif menu_choice == "4":
        break  # out of the loop
    else:
        print("Please only choose option 1, 2, 3, or 4")

print("Program Ended")
```

  Results from selecting option 4:

```
What would you like to do?:4
Program Ended

Process finished with exit code 0
```

  Results from selecting an option other than 1-4. In this case 11 was entered:

```
What would you like to do: 11
Please only choose option 1, 2, 3, or 4
```

Summary

By creating the Python program outlined in steps 1-7 above, a user will be presented with a menu of options upon running the program. The program allows the user to input a student's first and last name, along with the course they're registering for (option 1). As long as the first menu option is chosen, the user can continue to register as many students as needed. The user can then review the registration data (option 2) and save it to a .json file (option 3). When finished, there is an option for the user to exit the program (option 4). All of the registration data that was input will be saved to the .json file and continue to be available for review upon exiting and/or rerunning the program.

References

1. *MOD03-Notes.pdf page*
2. *MOD05-Notes .pdf page 13*
3. *MOD01-Notes.pdf page 28*
4. *MOD01-Notes.pdf page 32*
5. *MOD03-Notes.pdf page 14*
6. *MOD04-Notes.pdf page 29*
7. *MOD05-Notes .pdf page 17*
8. *MOD05-Notes .pdf page 19*
9. *MOD05-Notes .pdf page 20*
10. *MOD05-Notes .pdf page 24*
11. *MOD03-Notes.pdf page 16*
12. *MOD03-Notes.pdf page 17*
13. *MOD03-Notes.pdf page 8*
14. *MOD05-Notes .pdf page 3*
15. *MOD02-NOTES.pdf page 9*
16. *MOD02-Notes.pdf pages 17-18*
17. *MOD05-Notes .pdf page 16*